

LAZY LOADING IN REACT - COMPLETE NOTES

1. What is Lazy Loading?

- Lazy Loading is a technique to load components or modules only when they are needed.
- It helps to reduce the initial bundle size and improve the performance of the app.
- In React, we use `React.lazy()` and `Suspense` for Lazy Loading.

Load only
what you need,
when you need.



2. Why Lazy Loading?

- ✓ Reduces Initial Bundle Size
- ✓ Faster Initial Load
- ✓ Better Performance
- ✓ Improves User Experience
- ✓ Loads Code on Demand

Less Code
in Initial Load
= Faster App



3. Syntax

`React.lazy()` : Used to dynamically import a component.

`Suspense` : Used to show a fallback (loading UI) while the component is loading.

Basic Syntax

```
const MyComponent = React.lazy(() =>
  import('./MyComponent')
);
```

With Suspense

```
<Suspense fallback={<div>Loading...</div>}>
  <MyComponent />
</Suspense>
```

4. How it Works?



5. Example - Simple Lazy Loading

```
import React, { Suspense, lazy } from 'react';
// Lazy loaded component
const About = lazy(() => import('./About'));
const Home = () => <h2>Home Page</h2>;
function App() {
  return (
    <div>
      <h1>My React App</h1>
      <Suspense fallback={<h3>Loading...</h3>}>
        <About />
      </Suspense>
    </div>
  );
}
export default App;
```

About component
will be loaded
only when needed.



6. Example - Lazy Loading with Routes

```
import React, { Suspense, lazy } from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';

const Home = lazy(() => import('./pages/Home'));
const About = lazy(() => import('./pages/About'));

function App() {
  return (
    <BrowserRouter>
      <Suspense fallback={<h3>Loading...</h3>}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}
export default App;
```

Components in
routes are
loaded only
when route
is visited.



7. Lazy Loading Named Exports

```
const MyComponent = lazy(() =>
  import('./MyComponent').then(module => ({
    default: module.MyComponent
  })))
);
```

Use when
component is a
named export.



8. Lazy Loading Multiple Components

```
const CompA = lazy(() => import('./CompA'));
const CompB = lazy(() => import('./CompB'));
const CompC = lazy(() => import('./CompC'));

// Use in Suspense as needed
```

Load multiple
components
on demand.



9. Combining Lazy Loading with React.memo()

```
const HeavyComponent = React.lazy(() => import('./HeavyComponent'));
const MemoisedHeavy = React.memo(HeavyComponent);

// Use inside Suspense
```

Good for
performance
optimisation



10. Best Practices

- ✓ Lazy load components that are not needed on initial render.
- ✓ Always wrap lazy-loaded components with `<Suspense>`.
- ✓ Use a meaningful fallback UI (spinner, skeleton, loader).
- ✓ Avoid lazy loading small or frequently used components.
- ✓ Combine with `React.memo()`, `useCallback()`, `useMemo()` for best results.

Lazy Load
Smartly,
Not
Everything



11. Common Use Cases

- Large Components (Dashboards, Reports)
- Route based Code Splitting.
- Modal / Popup Components
- Admin Panels / Settings Pages
- Third-party heavy libraries or components

Use Lazy
Loading where
it really
matters.



12. Lazy Loading vs Code Splitting

Feature	Lazy Loading	Code Splitting
Definition	Load components on demand	Split code into smaller bundles
Purpose	Delay loading of components	Reduce initial bundle size
How	React.lazy() + Suspense	Dynamic import() + Build tools (Webpack, Vite)
Result	Component loaded when needed	App is split into multiple bundles
Example	Lazy load About component	Split vendor, route, or feature code

13. Things to Remember

- Lazy Loading is a form of Code Splitting.
- It helps to improve performance and user experience.
- Always provide a good fallback UI.
- Don't overuse Lazy Loading.

Better
Performance
= Happy
Users



14. Complete Lazy Loading Flow



Load on
Demand,
Not in
Advance.



15. Quick Revision

- ✓ `React.lazy()` → Load component dynamically.
- ✓ `Suspense` → Show fallback while loading.
- ✓ Reduces bundle size
- ✓ Improves performance
- ✓ Use for routes, modals, heavy components
- ✓ Provide meaningful fallback UI



Complete Flow

Import Component
with `React.lazy()`

Wrap with
`<Suspense fallback=...>`

Fallback UI
Shown

Component
Loads

Component
Rendered

16. Practical Example - Lazy Loaded Modal

```

const Modal = lazy(() => import('./components/Modal'));

function App(){
  const {open, setOpen} = useState(false);
  return (
    <div>
      <button onClick={() => setOpen(true)}>Open Modal</button>
      {open & (
        <Suspense fallback={<p>Loading Modal...</p>}>
          <Modal onClose={() => setOpen(false)} />
        </Suspense>
      )}
    </div>
  );
}
  
```

Modal
component
is loaded
only when
opened.



LAZY LOADING = SMALLER BUNDLE SIZE + FASTER LOAD TIME
+ BETTER USER EXPERIENCE



LOAD SMART, PERFORM FAST, DELIVER BETTER! ☆